SKILL DEVELOPMENT CELL

Day 1 – Web Hackathon

Problem Statement: BuildSpace – A Developer Collaboration Platform

In today's student developer ecosystem, collaboration is highly fragmented.
Professional profiles are managed on platforms like LinkedIn, code is hosted on GitHub, and team formation happens through informal communities such as Discord/Whatsapp.

This fragmentation creates challenges for students to:

- Find the right teammates
- Collaborate effectively on projects
- Discover relevant opportunities such as hackathons and open-source work

Challenge

Design and develop **BuildSpace**, a **responsive web application**, that brings together developer profiles, project collaboration, and opportunity discovery into a unified platform.

Objective

Build a platform that enables:

- Collaboration between student developers
- Discovery of projects and opportunities
- Active and engaging community interaction

Core Features (Minimum Requirements)

🔹 1. Developer Profiles

Users should be able to:

- Create and update profiles
- Add skills, interests, and projects

🔹 2. Project & Team Formation

Users should be able to:

- Create projects with descriptions and tech stacks
- Join or invite collaborators

🔹 3. Opportunity Board

Users can:

- Post or browse opportunities such as:
  - “Looking for teammates”
  - “Hiring for project roles”
  - “Hackathon team openings”

🔹 4. Interactive Feed / Dashboard

A dynamic feed displaying:

- New projects
- Team updates
- Opportunities

The platform should feel **interactive and community-driven**, not static

Optional Enhancements (Bonus)

- Dark/light mode
- Search & filtering (skills, roles, projects)
- Real-time updates / notifications
- Profile sharing

Technical Guidelines

Participants are free to use **any modern web technologies**.

Frontend:

- React / Next.js / Vue / Solid / HTML-CSS-JS

Backend (Optional but rewarded):

- Node.js / Express

- Firebase / Supabase
- Django / Flask

Database (Optional):

- Firebase Firestore
- MongoDB
- PostgreSQL

The choice of technology is flexible.
Evaluation will focus on **implementation, usability, and design**, not the tech stack.

Additional Requirements

- The application **must be responsive** (mobile + desktop compatible)
- Clean UI/UX with intuitive navigation
- Proper component structure and code organization

Deliverables

Participants must submit:

- **GitHub Repository** (source code)
- **Screen Recording Demo** (showing features and flow of the application)

Evaluation Criteria

- UI/UX Design — 30%
- Functionality — 25%
- Creativity & Innovation — 20%
- Technical Implementation — 15%
- Real-World Usefulness — 10%